



Technical University of Cluj-Napoca, Romania
Department of Computer Science

Programming Techniques

Implementing Concurrency in Java

Agenda

- Concurrency

Note: the information included in this presentation is taken from the references mentioned in the Bibliography section!

Concurrency

Overview

- Concurrent Software
 - Are software that can do more than one thing at a time
 - Examples of concurrent software
 - Application controlling the elevators in a building
 - Word processor - should always be ready to respond to keyboard and mouse events, no matter how busy it is reformatting text or updating the display
- Java platform is designed to support concurrent programming



Concurrency

Processes and Threads

- There are two basic units of execution in concurrent programming
 - Processes
 - Threads
- Process
 - Is an instance of program execution
- Thread
 - Represents a sequence of instructions which are executed within a process
 - Threads share the process's resources, including memory
 - Creating a new thread requires fewer resources than creating a new process

Concurrency

Processes and Threads

- Difference between processes and threads

- A program in execution is often referred as **process**. A **thread** is a subset(part) of the process.
- A **process** consists of multiple **threads**. A **thread** is a smallest part of the process that can execute concurrently with other parts(threads) of the process.
- A process is sometime referred as task. A thread is often referred as lightweight process
- A process has its own address space. A thread uses the process's address space and share it with the other threads of that process

Concurrency

Processes and Threads

- Java language supports multithreading through the **Thread** class as well as the **Runnable** interface available in the **java.lang** package

Concurrency

Threads - Overview

- Methods for creating a thread:
 - Defining a class that extends **Thread** class
 - Defining a class that implements **Runnable** interface



Concurrency

Threads - Overview

- Steps for creating a thread by extending **Thread** class
 - Define a class that extends **Thread** class
 - Overrides the **public void run()** method inherited from **Thread** class
 - Instantiates a thread object from the define class
 - Start the instantiated thread by calling the **start ()** method inherited from the **Thread** class
 - This method calls the **run()** method from **Thread** class

Concurrency

Threads - Overview

- Example for creating a Thread by extending the **Thread** class:

```
public class Fir
{
    public static void main(String args[])
    {
        FirdeExecutie fir=new FirdeExecutie();
        fir.start();
        System.out.println("Revenim la main");
    }
}

class FirdeExecutie extends Thread
{
    public void run()
    {
        for(int i=0;i<10;i++)
            System.out.println("Pasul "+i);
        System.out.println("Run s-a terminat");
    }
}
```

Concurrency

Threads - Overview

- Steps for creating a thread by implementing **Runnable** interface
 - Defines a class that implements **Runnable** interface
 - Implements **run()** method from the **Runnable**
 - Instantiates an object from the defined class
 - Instantiate a **Thread** object using a constructor from **Thread** class that takes as parameter an object of type **Runnable** (i.e. an instance of the class that implements the **Runnable** interface)
 - Start the thread by calling the **start()** method from **Thread** class
- Observation: Implementing **Runnable** interface is useful when the class of type **Thread** that need to be implemented already extends another class



Concurrency

Threads - Overview

```
public class Fir3
{
    public static void main(String args[])
    {
        FirdeExecutie fir=new FirdeExecutie();
        Thread thread=new Thread(fir);
        thread.start();
        System.out.println("Revenim la main");
    }
}

class A
{
    public void afis()
    {
        System.out.println("Este un exemplu simplu");
    }
}

class FirdeExecutie extends A implements Runnable
{
    public void run()
    {
        for(int i=0;i<5;i++)
            System.out.println("Pasul "+i);
        afis();
        System.out.println("Run s-a terminat");
    }
}
```



Concurrency

Threads – Life Cycle of a thread

- "New Thread" state

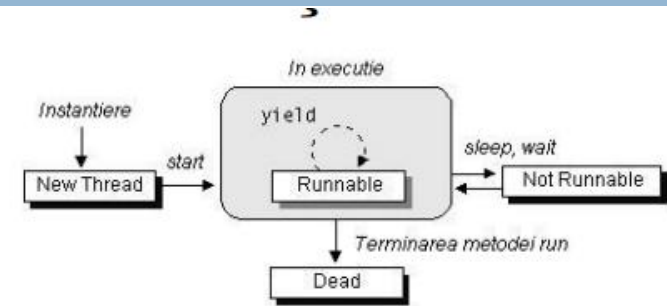
```
Thread fir = new Thread(object);  
// the thread is in "New Thread" state
```

- It doesn't allocate resources
- We have an thread instance, but we don't have an execution
- We can call **start()** method

- "Runnable" state

- The thread is in a runnable state when we call the start method on it
- The run method can be executed
- In this state the thread has allocated resources for execution

```
fir.start(); // the thread is in "Runnable" state
```



Concurrency

Threads – Life Cycle of a thread

- "Not Runnable" state

- The thread is in this state when some specific methods for **Thread** class are called (*i.e. sleep, wait – notify*)

```
try { // a pause for a second
```

```
    Thread.sleep(1000); }
```

```
catch (InterruptedException e) { ... }
```

- "Dead" state

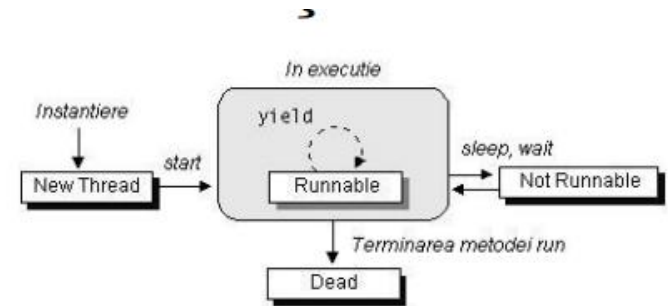
- The execution of the thread ends

- Execution of the threads must end naturally

```
public void run() { for(int i = a; i <= b; i += pas)  
    System.out.print(i + " " ); }
```

- Termination variables

```
public boolean executie = true;  
public void run() { while (executie) { ... } }
```



Concurrency

Thread Execution Order

- The Thread Scheduler is part of the JVM
 - Decide which thread to be executed at a specific time (from the set of eligible threads)
 - Removes the threads from the running state at the end of their execution
 - In the planning of the execution order there is a queue behavior
 - Once a thread has finished its piece of work, it is placed at the end of the queue where it expects to be executed again

Concurrency

Thread Execution Order

```
class NameRunnable implements Runnable {  
    public void run() {  
        for (int x = 0; x < 3; x++) {  
            System.out.println("Run by " + Thread.currentThread().getName() + ", x is " + x);  
        }  
    }  
}  
  
public class ManyNames {  
    public static void main(String [] args) {  
        NameRunnable nr = new NameRunnable();  
        Thread one = new Thread(nr);  
        Thread two = new Thread(nr);  
        Thread three = new Thread(nr);  
        one.setName("Fred"); two.setName("Lucy"); three.setName("Ricky");  
        one.start(); two.start(); three.start();  
    }  
}
```

Run by Lucy, x is 0
Run by Fred, x is 0
Run by Ricky, x is 0
Run by Fred, x is 1
Run by Fred, x is 2
Run by Lucy, x is 1
Run by Ricky, x is 1
Run by Lucy, x is 2
Run by Ricky, x is 2



Concurrency

Thread Execution Order

- Observation

- Inside a thread, things happen in a predictable order, but the actions of multiple threads can be mixed in an unexpected order
 - If the same code runs multiple times, or on different machines, the result may be different
- Even if we can not control the execution order, we can use same methods to influence this order

Concurrency

Thread Execution Order

- Example of methods that can be used to specify the execution order:

- From `java.lang.Thread`:
 - `public static void sleep(long millis)` throws `InterruptedException`
 - `public final void join()` throws `InterruptedException`
- From `java.lang.Object`:
 - `public final void wait()` throws `InterruptedException`
 - `public final void notify()`
 - `public final void notifyAll()`

Concurrency

Thread - Sleep

- `sleep()`
 - Static method from **Thread**
 - Causes the current thread to suspend execution for a specified period
 - The period is equal to the number of milliseconds given as an argument
 - Throws an exception of the **InterruptedException** type when a "sleepy" thread is interrupted

Concurrency

Threads – Pausing Execution with Sleep

```
public class SleepMessages {  
    public static void main(String args[])  
        throws InterruptedException {  
        String importantInfo[] = {  
            "Mares eat oats",  
            "Does eat oats",  
            "Little lambs eat ivy",  
            "A kid will eat ivy too"  
        };  
  
        for (int i = 0;  
            i < importantInfo.length;  
            i++) {  
            //Pause for 4 seconds  
            Thread.sleep(4000);  
            //Print a message  
            System.out.println(importantInfo[i]);  
        }  
    }  
}
```



Exception that *sleep* throws when another thread interrupts the current thread while sleep is active.

Concurrency

Threads – Interrupts

- A thread ends when the `run()` method finished
- Notify the thread that it should stop execution by using
 - `t.interrupt()`
- **`interrupt()`** does not cause the thread to finish - the method sets a **boolean** field in the thread data structure
- Java does not force the end of a thread when it is **interrupted**
- It's the thread's job of what it does when it's interrupted
- Interruption is a general mechanism to "get the attention" of the interrupted thread



Concurrency

Threads – Joins

- **join()**
 - It is a static method from **Thread**
 - Allows a thread to position itself at the end of another thread ("join on the end")
 - If a thread B can not do its job until thread A has been executed, then we want to "join" between threads A and B
 - This means that thread B become runnable after the thread A ends its execution
 - Ex.:

```
Thread t = new Thread();  
t.start();  
t.join(); // or t.join(500);
```
 - This code takes the thread into work and makes "join" with the thread t
 - join() with parameters means: the thread into work waits until *t* is complete, but if it lasts more than 500 ms, it gets runnable again

Concurrency

Threads – Synchronization (I)

- Problems with threads
 - **Thread interference** - happens when two operations, running in different threads, but acting on the same data, interleave $\Leftrightarrow$ the two operations consist of multiple steps, and the sequences of steps overlap
 - **Memory consistency errors** - different threads have inconsistent views of what should be the same data
- Threads synchronization – tool to prevent thread interference and memory consistency errors

Concurrency

Threads – Synchronization (II)

○ Problems with threads - example

```
class Buffer {
    private int number = -1;

    public int get() {
        return number;
    }

    public void put(int number) {
        this.number = number;
    }
}
```

```
class Producator extends Thread {
    private Buffer buffer;

    public Producator(Buffer b) {
        buffer = b;
    }

    public void run() {
        for (int i = 0; i < 10; i++) {
            buffer.put(i);
            System.out.println("Producatorul a pus:\t" + i);
            try {
                sleep((int)(Math.random() * 100));
            } catch (InterruptedException e) { }
        }
    }
}
```

Concurrency

Threads – Synchronization (III)

Problems with threads – example contd.

```
class Consumator extends Thread {
    private Buffer buffer;

    public Consumator(Buffer b) {
        buffer = b;
    }

    public void run() {
        int value = 0;
        for (int i = 0; i < 10; i++) {
            value = buffer.get();
            System.out.println("Consumatorul a primit:\t" + value);
        }
    }
}
```


Threads – Synchronization (IV)

Problems with threads – example contd.

```
public class TestSincronizare1 {
    public static void main(String[] args) {
        Buffer b = new Buffer();
        Producator p1 = new Producator(b);
        Consumator c1 = new Consumator(b);
        p1.start();
        c1.start();
    }
}
```

Producatorul a pus:	0
Consumatorul a primit:	0
Consumatorul a primit:	0
Consumatorul a primit:	0
Consumatorul a primit:	0
Consumatorul a primit:	0
Consumatorul a primit:	0
Consumatorul a primit:	0
Consumatorul a primit:	0
Consumatorul a primit:	0
Consumatorul a primit:	0
Producatorul a pus:	1
Producatorul a pus:	2
Producatorul a pus:	3
Producatorul a pus:	4
Producatorul a pus:	5
Producatorul a pus:	6
Producatorul a pus:	7
Producatorul a pus:	8
Producatorul a pus:	9

Concurrency

Threads – Synchronization (V)

- The problem is solved if at most one thread has access to that data at a moment
- Java implements mutual exclusion by specifying methods that share variables as being **synchronized**
- These methods must be of the same class
- Any thread that attempts to access a synchronized method of an object, as long as the method is used by another thread, is blocked until the first thread leaves the method

Concurrency

Threads – Synchronization (VI) - Example

```
import java.awt.*;
import java.applet.*;
import java.awt.event.*;
public class DoiContori extends Applet implements ActionListener
{
    private Button start;
    private int contor=0;
    public void init()
    {
        start=new Button("Start");
        add(start);
        start.addActionListener(this);
    }
    public void actionPerformed(ActionEvent event)
    {
        if(event.getSource()==start)
        {
            contor++;
            Graphics g=getGraphics();
            NumarPartajat numar=new NumarPartajat(g,contor);
            Contor1 contor1=new Contor1(numar);
            Contor2 contor2=new Contor2(numar);
            contor1.start();
            contor2.start();
        }
    }
}
```



Concurrency

Threads – Synchronization (V) -Example

```
class Contor1 extends Thread
{
    private NumarPartajat nr;
    public Contor1(NumarPartajat nr)
    {
        this.nr=nr;
    }
    public void run()
    {
        for(int i=1;i<=10;i++)
            nr.increment();
    }
}
```

```
class Contor2 extends Thread
{
    private NumarPartajat nr;
    public Contor2(NumarPartajat nr)
    {
        this.nr=nr;
    }
    public void run()
    {
        for(int i=1;i<=10;i++)
            nr.increment();
    }
}
```

```
class NumarPartajat
{
    private int n=0;
    private Graphics g;
    private int x=0;
    private int contor;
    public NumarPartajat(Graphics g,int contor)
    {
        this.g=g;
        this.contor=contor;
    }
    public synchronized void increment()
    {
        n=n+1;
        g.drawString(n+", ",x*20,30+15*contor);
        x++;
    }
}
```

Concurrency

Mutual exclusion and Synchronization -**wait()** and **notify()** methods

- Another problem of mutual exclusion is the producer-consumer example
 - The producer provides data that will be used by the consumer
 - What happens if the speed of the production and consumption is different ?
- For this we can use the methods **wait()**, **notify()** and **notifyAll()** from **Object** class
 - The **wait()** call will pass the called object to a blocked state
 - It remains blocked until you call **notify()** or **notifyAll()** methods for the same object
 - The necessary condition to call one of these methods is that the call to be made inside of the synchronized methods



Concurrency

Mutual exclusion and Synchronization

- For **wait()** method we can specify the waiting time
 - In this case the thread remains blocked until the time expires or another thread calls **notify()**
- Difference between **notify()** and **notifyAll()**
 - **notify()** is used to send the signal to a thread from the waiting list of the object
 - **notifyAll()** is used to send the signal to all the threads that are in the waiting state

Concurrency

Mutual exclusion and Synchronization

Calator.java

```
package Gara;
import java.util.*;
public class Calator{
    private long cID;
    private Date tsosire;
    private Date tplecare;
    public Calator( long cID, Date tsosire, Date tplecare ){
        this.cID = cID;
        this.tsosire = tsosire;
        this.tplecare = tplecare;
    }
    public long getCID(){
        return cID;
    }
    public Date getTsosire(){
        return tsosire;
    }
    public Date getTplecare(){
        return tplecare;
    }
    public String toString(){
        return ( Long.toString(cID)+" "+tsosire.toString()+" "+tplecare.toString());
    }
}
```

Concurrency

Mutual exclusion and Synchronization

Casa.java

```
package Gara;
import java.util.*;
public class Casa extends Thread{
    private Vector calatori=new Vector();

    public Casa( String name ){
        setName( name );
    }
    public void run(){
        try{
            while( true ){
                sterge_calator();
                sleep( (int) (Math.random()*4000) );
            }
        }
        catch( InterruptedException e ){
            System.out.println("Intrerupere");
            System.out.println( e.toString());
        }
    }

    public synchronized void adauga_calator( Calator c ) throws InterruptedException
    {
        calatori.addElement(c);
        notifyAll();
    }
    public synchronized void sterge_calator() throws InterruptedException
    {
        while( calatori.size() == 0 )
            wait();
        Calator c = ( Calator )calatori.elementAt(0);
        calatori.removeElementAt(0);
        System.out.println(Long.toString( c.getCID() )+" a fost deservit de casa "+getName());
        notifyAll();
    }
    public synchronized long lungime_coada() throws InterruptedException{
        notifyAll();
        long size = calatori.size();
        return size;
    }
}
```


Concurrency

Mutual exclusion and Synchronization

Prodicator.java

```
package Gara;
import java.util.*;
public class Prodicator extends Thread{

    private Casa casa[];
    private int nr_case;
    static long ID =0;
    private int nr_clienti;//Numar calatori care trebuie deserviti de catre casele de bilete
    public Prodicator( int nr_case, Casa casa[], String name, int nr_clienti ){
        setName( name );
        this.nr_case = nr_case;
        this.casa = new Casa[ nr_case ];
        this.nr_clienti = nr_clienti;
        for( int i=0; i<nr_case; i++){
            this.casa[ i ] = casa[ i ];
        }
    }
    private int min_index (){
        int index = 0;
        try
        {
            long min = casa[0].lungime_coada();
            for( int i=1; i<nr_case; i++){
                long lung = casa[ i ].lungime_coada();
                if ( lung < min ){
                    min = lung;
                    index = i;
                }
            }
        }
        catch( InterruptedException e ){
            System.out.println( e.toString());
        }
        return index;
    }

    public void run(){
        try
        {
            int i=0;
            while( i<nr_clienti ){
                i++;
                Calator c = new Calator( ++ID, new Date(), new Date() );
                int m = min_index();
                System.out.println("Calator :"+Long.toString( ID )+" adaugat la CASA "+
                Integer.toString(m));
                casa[ m ].adauga_calator( c );
                sleep( (int) (Math.random()*1000) );
            }
        }
        catch( InterruptedException e ){
            System.out.println( e.toString());
        }
    }
}
```

Concurrency

Mutual exclusion and Synchronization

TestGara.java

```
import Gara.*;
public class TestGara{
    public static void main( String args[] ){
        int i;
        int nr = 4;
        Casa c[] = new Casa[ nr ];
        for( i=0; i<nr; i++){
            c[ i ] = new Casa("Casa "+Integer.toString( i ));
            c[ i ].start();
        }

        Prodicator p = new Prodicator( nr , c, "Prodicator",30);
        p.start();
    }
}
```

Concurrency

Timer and TimerTask (I)

- **Java.util.Timer and TimerTask**

- Gives facility to schedule a task for future execution in a background thread
- Tasks may be scheduled for one-time execution, or for repeated execution at regular intervals
- **Timer** has got a **cancel** method using which all the scheduled tasks can be cancelled
- The actions of a **Timer** object are implemented as instances of the **TimerTask** class
 - **TimerTask** is an abstract class that implements the **Runnable** interface
 - We need to extend this class to create our own **TimerTask** that can be scheduled using the **Timer** class



Concurrency

Timer and TimerTask (II)

- Steps for using a timer
 - Create a subclass of the **TimerTask** class and overwrite the **run** method which will contain the action we want to plan
 - Create a thread by instantiating the **Timer** class
 - Create an object of type of subclass that extends **TimerTask**
 - Plan the execution of the object of subclass that extends **TimerTask** on using the **schedule** method from the **Timer** class

Concurrency

Timer and TimerTask (II)

- Example of methods from **Timer** class that can use to schedule repeated executions of tasks:
 - **schedule(TimerTask *task*, long *delay*, long *period*)**
 - **task** – This is the task to be scheduled.
 - **delay** – This is the delay in milliseconds before task is to be executed.
 - **period** – This is the time in milliseconds between successive task executions.
 - **schedule(TimerTask task, Date firstTime, long period)**
 - **task** – This is the task to be scheduled.
 - **firstTime** – This is the first time at which task is to be executed.
 - **period** – This is the time in milliseconds between successive task executions.

Concurrency

Timer and TimerTask (III)

```
import java.util.TimerTask;

public class RunMeTask extends TimerTask
{
    @Override
    public void run() {
        System.out.println("Run Me ~");
    }
}

import java.util.Timer;
import java.util.TimerTask;

public class App
{
    public static void main( String[] args )
    {
        TimerTask task = new RunMeTask();

        Timer timer = new Timer();
        timer.schedule(task, 1000, 60000);
    }
}
```

In this example, the timer will print the “Run Me ~” message every 60 seconds, with a 1 second delay for the first time of execution.



Concurrency- BlockingQueue

- `java.util.concurrent.BlockingQueue` interface
 - The methods from that interface make writing concurrent programs easier
 - Supports operations such as:
 - Wait for space to become available in the queue before storing an element
 - If the blocking queue reaches its upper limit, the producing thread is blocked while trying to insert the new object. It remains blocked until a consuming thread takes an object out of the queue.
 - Waiting for the queue to become non-empty before retrieving an element
 - If the consuming thread tries to take an object out of an empty queue, the consuming thread is blocked until a producing thread puts an object into the queue.



Concurrency

BlockingQueue

- BlockingQueue Interface
 - Has methods for inserting, removing and examining the elements in the queue
- Example of BlockingQueue Implementations
 - ArrayBlockingQueue
 - LinkedBlockinQueue

Concurrency

BlockingQueue

- Example of working with Java BlockingQueue

- The example uses the **ArrayBlockingQueue** implementation of the BlockingQueue interface
- The **BlockingQueueExample** class starts a **Producer** and a **Consumer** in separate threads
- The **Producer** inserts strings into a shared **BlockingQueue**, and the **Consumer** takes them out

Concurrency

BlockingQueue

```
public class BlockingQueueExample {  
    public static void main(String[] args) throws Exception {  
        BlockingQueue queue = new ArrayBlockingQueue(1024);  
  
        Producer producer = new Producer(queue);  
        Consumer consumer = new Consumer(queue);  
  
        new Thread(producer).start();  
        new Thread(consumer).start();  
  
        Thread.sleep(4000);  
    }  
}
```

```
public class Producer implements Runnable{  
    protected BlockingQueue queue = null;  
  
    public Producer(BlockingQueue queue) {  
        this.queue = queue;  
    }  
  
    public void run() {  
        try {  
            queue.put("1");  
            Thread.sleep(1000);  
            queue.put("2");  
            Thread.sleep(1000);  
            queue.put("3");  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

```
public class Consumer implements Runnable{  
    protected BlockingQueue queue = null;  
  
    public Consumer(BlockingQueue queue) {  
        this.queue = queue;  
    }  
  
    public void run() {  
        try {  
            System.out.println(queue.take());  
            System.out.println(queue.take());  
            System.out.println(queue.take());  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
    }  
}
```



Strategy Design Pattern

- **Strategy pattern** is a pattern that enables selecting an algorithm at runtime
- In Strategy pattern, we create objects which represent various strategies and a context object whose behavior varies as per its strategy object
 - The strategy object changes the executing algorithm of the context object

Strategy Design Example

Step 1

Create an interface.

Strategy.java

```
public interface Strategy {  
    public int doOperation(int num1, int num2);  
}
```

Step 3

Create Context Class.

Context.java

```
public class Context {  
    private Strategy strategy;  
  
    public Context(Strategy strategy){  
        this.strategy = strategy;  
    }  
  
    public int executeStrategy(int num1, int num2){  
        return strategy.doOperation(num1, num2);  
    }  
}
```

Step 2

Create concrete classes implementing the same interface.

OperationAdd.java

```
public class OperationAdd implements Strategy{  
    @Override  
    public int doOperation(int num1, int num2) {  
        return num1 + num2;  
    }  
}
```

OperationSubtract.java

```
public class OperationSubtract implements Strategy{  
    @Override  
    public int doOperation(int num1, int num2) {  
        return num1 - num2;  
    }  
}
```

OperationMultiply.java

```
public class OperationMultiply implements Strategy{  
    @Override  
    public int doOperation(int num1, int num2) {  
        return num1 * num2;  
    }  
}
```



Strategy Design Example

Step 4

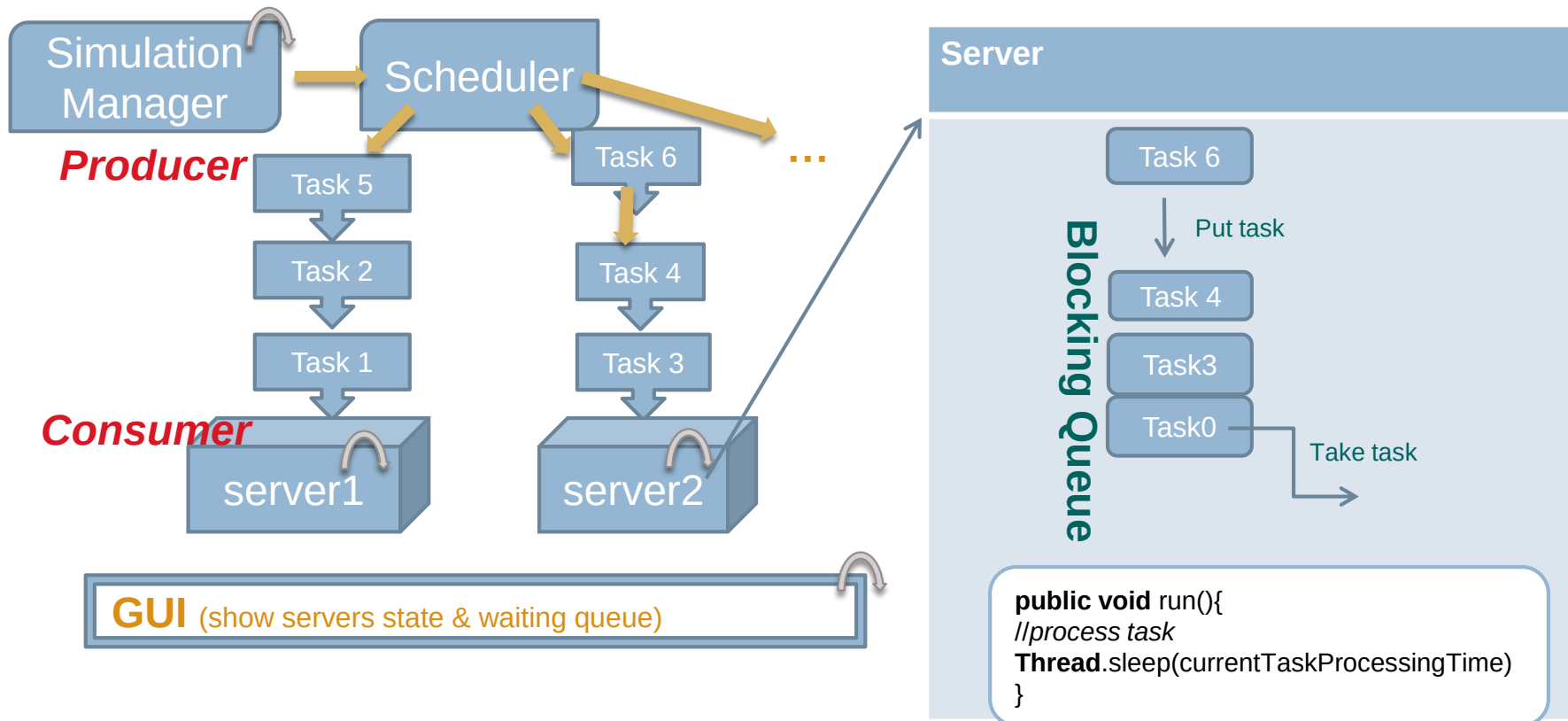
Use the *Context* to see change in behaviour when it changes its *Strategy*.

StrategyPatternDemo.java

```
public class StrategyPatternDemo {  
    public static void main(String[] args) {  
        Context context = new Context(new OperationAdd());  
        System.out.println("10 + 5 = " + context.executeStrategy(10, 5));  
  
        context = new Context(new OperationSubtract());  
        System.out.println("10 - 5 = " + context.executeStrategy(10, 5));  
  
        context = new Context(new OperationMultiply());  
        System.out.println("10 * 5 = " + context.executeStrategy(10, 5));  
    }  
}
```



Assignment 2.



Assignment 2

Task

- Modeled using:
 - arrivalTime
 - finishTime
 - processingPeriod
- $\text{FinishTime} = \text{arrivalTime} + \text{processingPeriod} + \text{waitingPeriodOnChosenServer}$

```
public class Task {  
  
    private int arrivalTime;  
    private int processingTime;  
    ....  
}
```

Assignment 2

Server -- Runnable

- Modeled using:
 - Tasks (BlockingQueue<Task>...
 - WaitingPeriod (AtomicInteger –
 - decremented by current thread once a task is completed
 - incremented by scheduler thread adding new tasks)

```
public class Server implements Runnable {
    private BlockingQueue<Task> tasks;
    private AtomicInteger waitingPeriod;

    public Server() {
        //initialize queue and waitingPeriod
    }

    public void addTask(Task newTask) {
        //add task to queue
        //increment the waitingPeriod
    }

    public void run() {
        while (true) {
            //take next task from queue
            // stop the thread for a time equal with the task's processing time
            // decrement the waitingPeriod
        }
    }

    public Task[] getTasks() {
        ...
    }
}
```


Assignment 2

○ Scheduler

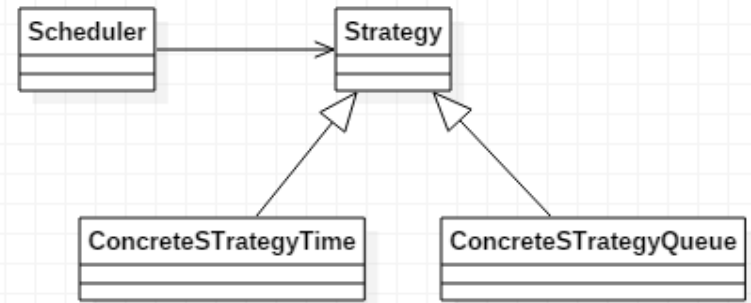
- Sends tasks to Servers according to the established **strategy**
- Modeled Using :
 - Servers
 - Constraints:
 - maxNoServers,
 - maxLoadPerServer

```
public class Scheduler {  
  
    private List<Server> servers;  
    private int maxNoServers;  
    private int maxTasksPerServer;  
    private Strategy strategy;  
  
    public Scheduler(int maxNoServers, int maxTasksPerServer) {  
        //for maxNoServers  
        // - create server object  
        // - create thread with the object  
    }  
  
    public void changeStrategy(SelectionPolicy policy){  
        //apply strategy patter to instantiate the strategy with the concrete  
        //strategy corresponding to policy  
        if(policy == SelectionPolicy.SHORTEST_QUEUE){  
            strategy = new ConcreteStrategyQueue();  
        }  
        if(policy == SelectionPolicy.SHORTEST_TIME){  
            strategy = new ConcreteStrategyTime();  
        }  
    }  
  
    public void dispatchTask(Task t) {  
        //call the strategy addTask method  
    }  
  
    public List<Server> getServers() {  
        return servers;  
    }  
}
```

Assignment 2

- **Scheduler – Strategy Patter**
- Choose the policy to distribute clients
- https://en.wikipedia.org/wiki/Strategy_pattern

```
public interface Strategy {  
    public void addTask(List<Server> servers, Task t);  
}
```



```
public enum SelectionPolicy {  
    SHORTEST_QUEUE, SHORTEST_TIME  
}  
  
public class ConcreteStrategyTime implements Strategy {  
  
    @Override  
    public void addTask(List<Server> servers, Task t) {  
        // TODO Auto-generated method stub  
    }  
}
```

Assignment 2

Simulation Manager-Runnable

- Generates randomly the tasks with:
 - Arrival time
 - processingPeriod
- Contains simulation loop:
 - CurrentTime
 - Call scheduler to dispatch task
 - Update UI

```
public class SimulationManager implements Runnable{
    //data read from UI
    public int timeLimit = 100; //maximum processing time - read from UI
    public int maxProcessingTime = 10;
    public int minProcessingTime = 2;
    public int numberOfServers = 3;
    public int numberOfClients = 100;
    public SelectionPolicy selectionPolicy = SelectionPolicy.SHORTEST_TIME;

    //entity responsible with queue management and client distribution
    private Scheduler scheduler;
    //frame for displaying simulation
    private SimulationFrame frame;
    //pool of tasks (client shopping in the store)
    private List<Task> generatedTasks;

    public SimulationManager(){
        // initialize the scheduler
        //     => create and start numberOfServers threads
        //     => initialize selection strategy => createStrategy
        // initialize frame to display simulation
        // generate numberOfClients clients using generateNRandomTasks()
        //and store them to generatedTasks
    }

    private void generateNRandomTasks(){
        // generate N random tasks:
        // - random processing time
        //minProcessingTime < processingTime < maxProcessingTime
        // - random arrivalTime
        //sort list with respect to arrivalTime
    }
}
```

Assignment 2

Simulation Manager-Runnable

- Generates randomly the tasks with:
 - Arrival time
 - processingPeriod
- Contains simulation loop:
 - CurrentTime
 - Call scheduler to dispatch tasks
 - Update UI

```
@Override
public void run() {
    int currentTime = 0;
    while (currentTime < timeLimit){
        // iterate generatedTasks list and pick tasks that have the
        // arrivalTime equal with the currentTime
        // - send task to queue by calling the dispatchTask method
        // from Scheduler
        // - delete client from list
        // update UI frame
        currentTime++;
        // wait an interval of 1 second
    }
}

public static void main(String[] args){
    SimulationManager gen = new SimulationManager();
    Thread t = new Thread(gen);
    t.start();
}
```

Assignment 2

○ SimulatorFrame

- A panel containing a JScrollPane corresponding to each server
- Each JScrollPane should contain the JList of tasks corresponding to the server

```
public class SimulatorFrame extends JFrame {
    private static final long serialVersionUID = 1L;
    private JPanel panel;
    private int WIDTH = 600, HEIGHT = 600;

    public SimulatorFrame() {
        panel = new JPanel();
        this.add(panel);
        this.setSize(WIDTH, HEIGHT);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        this.setLocationRelativeTo(null);
        this.setVisible(true);
    }

    public void displayData(Task[][] tasks) {
        //remove all previous elements from panel and revalidate panel
        // add a JScrollPane with a JList containing tasks
        // add scrollPane to panel
        // repaint and revalidate panel
    }
}
```

Bibliography

- **Programming Techniques Courses**
 - http://coned.utcluj.ro/~salomie/PT_Lic/
- **Swing**
 - <http://docs.oracle.com/javase/tutorial/uiswing/index.html>
- **Java Concurrency**
 - <http://docs.oracle.com/javase/tutorial/essential/concurrency/>
 - <http://docs.oracle.com/javase/7/docs/api/java/util/Timer.html>
 - <http://docs.oracle.com/javase/7/docs/api/java/util/TimerTask.html>
 - <http://www.journaldev.com/1050/java-timer-and-timertask-example-tutorial>
 - https://users.info.uvt.ro/~iordan/P_III/Cristian_Frasinaru-Curs_practic_de_Java.pdf
 - <https://www.youtube.com/watch?v=YdlnEWC-7Wo>
 - <http://tutorials.jenkov.com/java-util-concurrent/blockingqueue.html>
- https://www.tutorialspoint.com/design_pattern/strategy_pattern.htm